

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers

# Load the image
image_path = '/content/leaf.jpg'
image = cv2.imread(image_path)
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Resize image to the desired input size of the model
target_size = (256, 256) # Changed to match model input size
image_resized = cv2.resize(image_rgb, target_size)

```

```

# Normalize the image
image_normalized = image_resized / 255.0
# Add batch dimension
image_batch = np.expand_dims(image_normalized, axis=0)
# UNet Model Definition (You can also use a pre-trained model)
def build_unet_model(input_size=(256, 256, 3)):
    inputs = layers.Input(input_size)
# Encoder
    c1 = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(inputs)
    c1 = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(c1)
    p1 = layers.MaxPooling2D((2, 2))(c1)
    c2 = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(p1)
    c2 = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(c2)
    p2 = layers.MaxPooling2D((2, 2))(c2)

    c3 = layers.Conv2D(128, (3, 3), activation='relu', padding='same')(p2)
    c3 = layers.Conv2D(128, (3, 3), activation='relu', padding='same')(c3)
    p3 = layers.MaxPooling2D((2, 2))(c3)
# Bottleneck
    c4 = layers.Conv2D(256, (3, 3), activation='relu', padding='same')(p3)
    c4 = layers.Conv2D(256, (3, 3), activation='relu', padding='same')(c4)

# Decoder
    u5 = layers.Conv2DTranspose(128, (2, 2), strides=(2, 2), padding='same')(c4)
    u5 = layers.concatenate([u5, c3])
    c5 = layers.Conv2D(128, (3, 3), activation='relu', padding='same')(u5)
    c5 = layers.Conv2D(128, (3, 3), activation='relu', padding='same')(c5)

    u6 = layers.Conv2DTranspose(64, (2, 2), strides=(2, 2), padding='same')(c5)
    u6 = layers.concatenate([u6, c2])
    c6 = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(u6)
    c6 = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(c6)
    u7 = layers.Conv2DTranspose(32, (2, 2), strides=(2, 2), padding='same')(c6)
    u7 = layers.concatenate([u7, c1])
    c7 = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(u7)
    c7 = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(c7)
    outputs = layers.Conv2D(1, (1, 1), activation='sigmoid')(c7)
    model = tf.keras.Model(inputs=[inputs], outputs=[outputs])
    return model
model = build_unet_model()
model.summary()

```

[Show hidden output](#)

```

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
# Example: model.fit(x_train, y_train, epochs=10, batch_size=2, validation_split=0.1)
# For demonstration, we will assume the model is already trained.

```

```

x_train = np.random.rand(5, 256, 256, 3)
y_train = np.random.randint(0, 2, (5, 256, 256, 1))
model.fit(x_train, y_train, epochs=50, batch_size=1, validation_split=0.2, verbose=1)

```

[Show hidden output](#)

```

# Make a prediction
prediction = model.predict(image_batch)
# Post-process the prediction

```

```
predicted_mask = np.squeeze(prediction) # Remove batch dimension
predicted_mask = np.round(predicted_mask) # Binarize the mask
# Display the result
plt.figure(figsize=(12, 6))
plt.title('(2311cs010480) - N.Rishik',loc='right')
plt.subplot(1, 3, 1)
plt.title('Original Image')
plt.imshow(image_rgb)
plt.axis('off')
plt.subplot(1, 3, 2)
plt.title('Resized Image')
plt.imshow(image_resized)
plt.axis('off')
plt.subplot(1, 3, 3)
plt.title('Predicted Mask')
plt.imshow(predicted_mask, cmap='gray')
plt.axis('off')
plt.show()
```

[Show hidden output](#)